



MediCare+

Sistema de Agendamento de Consultas Médicas

Apostila Completa do Curso

Desenvolvimento Desktop com Python

Aulas 1, 2 e 3 · Projeto Final Completo

Aula 1

MVC + MySQL

Aula 2

ttkbootstrap

Aula 3

Projeto Final

Sumário

INTRODUÇÃO AO PROJETO MediCare+	3
AULA 1 – ORGANIZANDO O SISTEMA COM MVC + MYSQL	5
1.1 O que é MVC e por que usamos?	5
1.2 Estrutura de Pastas do MediCare+	7
1.3 Configurando o Banco de Dados MySQL	9
1.4 Model: Comunicação com o Banco	12
1.5 Controller: As Regras de Negócio	16
1.6 Exercícios da Aula 1	20
AULA 2 – INTERFACE PROFISSIONAL COM ttkbootstrap	22
2.1 O que é ttkbootstrap?	22
2.2 Temas e Widgets Estilizados	23
2.3 Criando o Menu Lateral (Dashboard)	26
2.4 Formulários Bonitos e Responsivos	29
2.5 Exercícios da Aula 2	34
AULA 3 – PROJETO FINAL COMPLETO	36
3.1 Tela de Login	36
3.2 Cadastro de Clientes (Pacientes)	39
3.3 Cadastro de Médicos	43
3.4 Cadastro de Planos de Saúde	47
3.5 Tela de Agendamento	51
3.6 Listagem e Filtros	57
3.7 Executando o Sistema Completo	60
APÊNDICE – Referência Rápida de Comandos	62

Introdução ao Projeto MediCare+

Bem-vindo ao curso! Nas próximas aulas você vai construir do zero o **MediCare+**, um sistema profissional de agendamento de consultas médicas usando Python.

PROJETO MediCare+

O MediCare+ vai ter as seguintes funcionalidades:

- Cadastro de Pacientes (clientes)
- Cadastro de Médicos (com especialidade)
- Cadastro de Planos de Saúde
- Agendamento de Consultas (com data e horário)
- Listagem e pesquisa de agendamentos
- Interface moderna e profissional com ttkbootstrap

O que você vai aprender

O curso está dividido em 3 aulas progressivas:

Aula	Tema	O que aprende
1	MVC + MySQL	Organizar o código em camadas, criar e conectar banco de dados
2	ttkbootstrap	Criar telas modernas, usar temas e widgets estilizados
3	Projeto Final	Montar o sistema completo MediCare+ do zero ao toque final

Pré-requisitos

O que você precisa ter instalado antes de começar:

- ☒ Python 3.10 ou superior
- ☒ MySQL Server (ou XAMPP com MySQL)
- ☒ VS Code ou qualquer editor de código
- ☒ Noção básica de Tkinter (Labels, Entry, Button, pack/grid)

Instalações necessárias

Abra o terminal (CMD no Windows) e digite os comandos abaixo:

CÓDIGO

```
# Instalar as bibliotecas necessárias
pip install pymysql
pip install ttkbootstrap

# Verificar se foram instaladas com sucesso
python -c "import pymysql; print('PyMySQL OK')"
python -c "import ttkbootstrap; print('ttkbootstrap OK')"
```

DICA DO PROFESSOR

Se der erro ao instalar, tente: `pip install --upgrade pip`
Depois tente instalar novamente: `pip install pymysql ttkbootstrap`

Aula 1 – Organizando o Sistema com MVC + MySQL

1.1 O que é MVC e por que usamos?

Antes de escrever qualquer código do MediCare+, precisamos entender o padrão **MVC**. Esse padrão é usado em projetos reais no mercado de trabalho.

MVC significa:

- M → Model (Modelo): tudo que tem a ver com os DADOS e o banco de dados
- V → View (Visão): tudo que o usuário VÊ – as telas
- C → Controller (Controlador): as REGRAS DE NEGÓCIO – o que acontece quando clica

Analogia do Restaurante

Pense em um restaurante:

Restaurante	MVC	No MediCare+
 Cardápio (o que o cliente vê)	View	Tela de Cadastro, tela de Agendamento
 Cozinheiro (quem processa)	Controller	Valida se o campo está vazio, chama o Model
 Geladeira / Estoque (onde guarda)	Model	Salva paciente no MySQL, busca médicos

Por que separar em camadas?

Sem MVC (código bagunçado):

CÓDIGO

```
# ✗ ERRADO - tudo misturado na mesma função
def salvar():
    nome = entry_nome.get()           # busca da tela (View)
    if nome == '':                   # validação (Controller)
        messagebox.showerror('Erro!')
    conn = pymysql.connect(...)      # banco de dados (Model)
    cursor = conn.cursor()
    cursor.execute('INSERT ...')
    # Impossível reaproveitar, impossível de manter!
```

Com MVC (organizado):

 CÓDIGO

```
# ☒ CORRETO - cada coisa no seu lugar

# View (cliente_view.py) - só captura o que o usuário digitou
def ao_clicar_salvar():
    nome = entry_nome.get()
    resultado = ClienteController.salvar(nome)
    messagebox.showinfo('Resultado', resultado)

# Controller (cliente_controller.py) - só valida
def salvar(nome):
    if nome.strip() == '':
        return 'Nome é obrigatório!'
    return ClienteModel.inserir(nome)

# Model (cliente_model.py) - só fala com o banco
def inserir(nome):
    cursor.execute('INSERT INTO pacientes (nome) VALUES (%s)', (nome,))
```

 DICA DO PROFESSOR

Pense assim: cada camada tem UMA responsabilidade.

View = capturar dados da tela.

Controller = validar os dados.

Model = salvar/buscar no banco de dados.

1.2 Estrutura de Pastas do MediCare+

Antes de criar qualquer arquivo Python, vamos criar a estrutura de pastas.

 CÓDIGO

```
medicare_plus/           ← pasta raiz do projeto
├── main.py               ← arquivo que INICIA o sistema
├── database/
│   └── conexao.py        ← configuração da conexão MySQL
├── model/
│   ├── paciente_model.py
│   ├── medico_model.py
│   ├── plano_model.py
│   └── agendamento_model.py
├── controller/
│   ├── paciente_controller.py
│   └── medico_controller.py
```

```
|   | plano_controller.py
|   | agendamento_controller.py
|   |
|   | view/
|   | | main_view.py      ← tela principal (menu)
|   | | paciente_view.py
|   | | medico_view.py
|   | | plano_view.py
|   | | agendamento_view.py
```

REGRA DE OURO – muito importante!

NUNCA execute os arquivos de dentro das pastas (view, model, controller).
SEMPRE execute apenas o main.py!
Motivo: os imports usam o caminho relativo a partir do main.py.
Rodar um arquivo interno causa erro de importação.

Como criar a estrutura de pastas

No terminal, dentro da pasta do projeto:

CÓDIGO

```
# Windows (CMD)
mkdir medicare_plus
cd medicare_plus
mkdir database model controller view

# Mac/Linux
mkdir -p medicare_plus/database medicare_plus/model
mkdir -p medicare_plus/controller medicare_plus/view
```

Criando os arquivos `__init__.py`

Todo Python que funciona como pacote (pasta com módulos) precisa de um arquivo chamado `__init__.py` (pode estar vazio).

CÓDIGO

```
# Crie um arquivo vazio __init__.py em cada pasta:

medicare_plus/database/__init__.py ← arquivo vazio
medicare_plus/model/__init__.py   ← arquivo vazio
medicare_plus/controller/__init__.py ← arquivo vazio
medicare_plus/view/__init__.py    ← arquivo vazio
```

1.3 Configurando o Banco de Dados MySQL

Agora vamos criar o banco de dados e as tabelas no MySQL.

Passo 1 – Criar o banco de dados

Abra o MySQL Workbench (ou o terminal do MySQL) e execute:

CÓDIGO

```
-- Execute no MySQL Workbench ou terminal MySQL

CREATE DATABASE IF NOT EXISTS medicare_db
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

-- Seleciona o banco para usar
USE medicare_db;
```

Passo 2 – Criar as tabelas

CÓDIGO

```
-- Tabela de Planos de Saúde (criamos primeiro pois Paciente depende dela)
CREATE TABLE IF NOT EXISTS planos (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    nome        VARCHAR(100) NOT NULL,
    codigo      VARCHAR(20)  NOT NULL,
    ativo       TINYINT(1)   DEFAULT 1
);

-- Tabela de Médicos
CREATE TABLE IF NOT EXISTS medicos (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    nome        VARCHAR(100) NOT NULL,
    especialidade VARCHAR(80)  NOT NULL,
    crm         VARCHAR(20)  NOT NULL UNIQUE,
    ativo       TINYINT(1)   DEFAULT 1
);

-- Tabela de Pacientes
CREATE TABLE IF NOT EXISTS pacientes (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    nome        VARCHAR(100) NOT NULL,
    cpf         VARCHAR(14)  NOT NULL UNIQUE,
    telefone    VARCHAR(20),
    email       VARCHAR(100),
    plano_id    INT,
    FOREIGN KEY (plano_id) REFERENCES planos(id)
);

-- Tabela de Agendamentos
CREATE TABLE IF NOT EXISTS agendamentos (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    paciente_id INT NOT NULL,
```



```
medico_id    INT NOT NULL,
data_hora    DATETIME NOT NULL,
status       ENUM('agendado','cancelado','realizado') DEFAULT
'agendado',
observacao   TEXT,
FOREIGN KEY (paciente_id) REFERENCES pacientes(id),
FOREIGN KEY (medico_id)   REFERENCES medicos(id)
);
```

Passo 3 – Dados de exemplo para testes

CÓDIGO

```
-- Insira alguns dados para poder testar o sistema
INSERT INTO planos (nome, codigo) VALUES
('Unimed',      'UNI-001'),
('Bradesco Saúde', 'BRA-002'),
('Particular',  'PAR-000');

INSERT INTO medicos (nome, especialidade, crm) VALUES
('Dr. Carlos Silva',  'Clínico Geral',  'CRM-123456'),
('Dra. Ana Oliveira', 'Cardiologia',    'CRM-789012'),
('Dr. Pedro Santos',  'Ortopedia',      'CRM-345678');
```

Passo 4 – Arquivo de Conexão (database/conexao.py)

Esse arquivo é o 'coração' que liga nosso Python ao MySQL:

CÓDIGO

```
# database/conexao.py
import pymysql

# ——— CONFIGURAÇÕES ———
# Altere abaixo com os dados do SEU MySQL
HOST      = 'localhost'
USUARIO   = 'root'
SENHA     = ''           # sua senha do MySQL
BANCO     = 'medicare_db'
# ———

def obter_conexao():
    """
    Cria e retorna uma conexão com o banco MySQL.
    Sempre que precisar do banco, chame essa função.
    """
    conexao = pymysql.connect(
        host=HOST,
        user=USUARIO,
        password=SENHA,
        database=BANCO,
```

```
        charset='utf8mb4',
        cursorclass=pymysql.cursors.DictCursor # retorna dicionários
    )
    return conexao

def testar_conexao():
    """Testa se a conexão está funcionando. Use para depurar."""
    try:
        conn = obter_conexao()
        print('☑ Conexão com MySQL OK!')
        conn.close()
    except Exception as e:
        print(f'✗ Erro na conexão: {e}')

# Teste ao rodar o arquivo diretamente
if __name__ == '__main__':
    testar_conexao()
```

O que é DictCursor?

Normalmente o PyMySQL retorna os dados como uma tupla: ('Carlos', 'Clínico Geral')
Com DictCursor, retorna como dicionário: {'nome': 'Carlos', 'especialidade': 'Clínico Geral'}
Muito mais fácil de usar! Você acessa pelo nome da coluna.

1.4 Model – Comunicação com o Banco

Os arquivos de Model são os únicos que falam diretamente com o MySQL. Eles executam os comandos SQL (SELECT, INSERT, UPDATE, DELETE).

Paciente Model (model/paciente_model.py)

CÓDIGO

```
# model/paciente_model.py
from database.conexao import obter_conexao

class PacienteModel:

    @staticmethod
    def inserir(nome, cpf, telefone, email, plano_id):
        """Insere um novo paciente no banco de dados."""
        conn = obter_conexao()
        try:
            with conn.cursor() as cursor:
                sql = """
                    INSERT INTO pacientes (nome, cpf, telefone, email,
plano_id)
```

```
        VALUES (%s, %s, %s, %s, %s)
        """
        # %s são placeholders de segurança (evita SQL Injection!)
        cursor.execute(sql, (nome, cpf, telefone, email, plano_id))
        conn.commit()    # confirma a gravação no banco
        return True
    except Exception as e:
        conn.rollback()  # desfaz em caso de erro
        raise e
    finally:
        conn.close()     # SEMPRE feche a conexão

    @staticmethod
    def listar_todos():
        """Retorna todos os pacientes com nome do plano."""
        conn = obter_conexao()
        try:
            with conn.cursor() as cursor:
                sql = """
                SELECT p.*, pl.nome AS nome_plano
                FROM pacientes p
                LEFT JOIN planos pl ON p.plano_id = pl.id
                ORDER BY p.nome
                """
                cursor.execute(sql)
                return cursor.fetchall()  # retorna lista de dicionários
        finally:
            conn.close()

    @staticmethod
    def buscar_por_id(paciente_id):
        """Retorna um paciente específico pelo ID."""
        conn = obter_conexao()
        try:
            with conn.cursor() as cursor:
                cursor.execute('SELECT * FROM pacientes WHERE id = %s',
(paciente_id,))
                return cursor.fetchone()  # retorna apenas um registro
        finally:
            conn.close()

    @staticmethod
    def atualizar(paciente_id, nome, cpf, telefone, email, plano_id):
        """Atualiza os dados de um paciente."""
        conn = obter_conexao()
        try:
            with conn.cursor() as cursor:
                sql = """
                UPDATE pacientes
                SET nome=%s, cpf=%s, telefone=%s, email=%s, plano_id=%s
                WHERE id = %s
                """
                cursor.execute(sql, (nome, cpf, telefone, email, plano_id,
paciente_id))
```

```
        conn.commit()
        return True
    except Exception as e:
        conn.rollback()
        raise e
    finally:
        conn.close()

    @staticmethod
    def deletar(paciente_id):
        """Remove um paciente do banco."""
        conn = obter_conexao()
        try:
            with conn.cursor() as cursor:
                cursor.execute('DELETE FROM pacientes WHERE id = %s',
(paciente_id,))
            conn.commit()
        finally:
            conn.close()
```

Por que @staticmethod?

Usamos @staticmethod porque esses métodos não precisam de um objeto criado.

Em vez de: modelo = PacienteModel() → modelo.inserir(...)

Chamamos direto: PacienteModel.inserir(...)

Mais simples para nosso nível de projeto!

Médico Model (model/medico_model.py)

CÓDIGO

```
# model/medico_model.py
from database.conexao import obter_conexao

class MedicoModel:

    @staticmethod
    def inserir(nome, especialidade, crm):
        conn = obter_conexao()
        try:
            with conn.cursor() as cursor:
                sql = 'INSERT INTO medicos (nome, especialidade, crm)
VALUES (%s, %s, %s)'
                cursor.execute(sql, (nome, especialidade, crm))
            conn.commit()
            return True
        except pymysql.err.IntegrityError:
            # CRM duplicado - o banco rejeita pois é UNIQUE
            raise ValueError('CRM já cadastrado no sistema!')
        finally:
```

```
        conn.close()

    @staticmethod
    def listar_todos():
        conn = obter_conexao()
        try:
            with conn.cursor() as cursor:
                cursor.execute('SELECT * FROM medicos WHERE ativo=1 ORDER
BY nome')
            return cursor.fetchall()
        finally:
            conn.close()

    @staticmethod
    def listar_nomes_ids():
        """Retorna lista simples (id, nome) para preencher Combobox."""
        conn = obter_conexao()
        try:
            with conn.cursor() as cursor:
                cursor.execute('SELECT id, nome FROM medicos WHERE ativo=1
ORDER BY nome')
            return cursor.fetchall()
        finally:
            conn.close()
```

Plano Model (model/plano_model.py)

CÓDIGO

```
# model/plano_model.py
from database.conexao import obter_conexao

class PlanoModel:

    @staticmethod
    def inserir(nome, codigo):
        conn = obter_conexao()
        try:
            with conn.cursor() as cursor:
                cursor.execute(
                    'INSERT INTO planos (nome, codigo) VALUES (%s, %s)',
                    (nome, codigo)
                )
            conn.commit()
            return True
        finally:
            conn.close()

    @staticmethod
    def listar_todos():
        conn = obter_conexao()
        try:
```

```
        with conn.cursor() as cursor:
            cursor.execute('SELECT * FROM planos WHERE ativo=1 ORDER BY
nome')
            return cursor.fetchall()
    finally:
        conn.close()
```

Agendamento Model (model/agendamento_model.py)

CÓDIGO

```
# model/agendamento_model.py
from database.conexao import obter_conexao

class AgendamentoModel:

    @staticmethod
    def inserir(paciente_id, medico_id, data_hora, observacao=''):
        conn = obter_conexao()
        try:
            with conn.cursor() as cursor:
                sql = """
                    INSERT INTO agendamentos
                        (paciente_id, medico_id, data_hora, observacao)
                    VALUES (%s, %s, %s, %s)
                """
                cursor.execute(sql, (paciente_id, medico_id, data_hora,
observacao))
            conn.commit()
            return True
        finally:
            conn.close()

    @staticmethod
    def listar_todos():
        """Lista todos os agendamentos com JOIN nas tabelas
relacionadas."""
        conn = obter_conexao()
        try:
            with conn.cursor() as cursor:
                sql = """
                    SELECT
                        a.id,
                        p.nome AS paciente,
                        m.nome AS medico,
                        m.especialidade,
                        a.data_hora,
                        a.status,
                        a.observacao
                    FROM agendamentos a
                    JOIN pacientes p ON a.paciente_id = p.id
                    JOIN medicos    m ON a.medico_id    = m.id
                """
                cursor.execute(sql)
                return cursor.fetchall()
        finally:
            conn.close()
```

```
        ORDER BY a.data_hora DESC
        """
        cursor.execute(sql)
        return cursor.fetchall()
    finally:
        conn.close()

    @staticmethod
    def verificar_conflito(medico_id, data_hora):
        """Verifica se o médico já tem consulta neste horário."""
        conn = obter_conexao()
        try:
            with conn.cursor() as cursor:
                sql = """
                SELECT COUNT(*) AS total FROM agendamentos
                WHERE medico_id = %s
                AND data_hora = %s
                AND status = 'agendado'
                """
                cursor.execute(sql, (medico_id, data_hora))
                resultado = cursor.fetchone()
                return resultado['total'] > 0 # True se tiver conflito
        finally:
            conn.close()
```

1.5 Controller – As Regras de Negócio

O Controller valida os dados antes de enviá-los ao Model. Ele é o 'fiscal' do sistema.

Paciente Controller (controller/paciente_controller.py)

CÓDIGO

```
# controller/paciente_controller.py
from model.paciente_model import PacienteModel

class PacienteController:

    @staticmethod
    def salvar(nome, cpf, telefone, email, plano_id):
        """Valida os dados e chama o Model para inserir."""

        # — VALIDAÇÕES —————
        if not nome.strip():
            return False, 'Nome do paciente é obrigatório!'

        if not cpf.strip():
            return False, 'CPF é obrigatório!'

        if len(cpf.replace('.', '').replace('-', '')) != 11:
            return False, 'CPF deve ter 11 dígitos!'

        # —————
```

```
        try:
            PacienteModel.inserir(nome, cpf, telefone, email, plano_id)
            return True, 'Paciente cadastrado com sucesso!'
        except Exception as e:
            return False, f'Erro ao cadastrar: {e}'

    @staticmethod
    def listar():
        """Retorna todos os pacientes."""
        return PacienteModel.listar_todos()

    @staticmethod
    def atualizar(paciente_id, nome, cpf, telefone, email, plano_id):
        if not nome.strip():
            return False, 'Nome é obrigatório!'
        try:
            PacienteModel.atualizar(paciente_id, nome, cpf, telefone,
            email, plano_id)
            return True, 'Paciente atualizado com sucesso!'
        except Exception as e:
            return False, f'Erro: {e}'
```

Agendamento Controller (controller/agendamento_controller.py)

CÓDIGO

```
# controller/agendamento_controller.py
from model.agendamento_model import AgendamentoModel
from datetime import datetime

class AgendamentoController:

    @staticmethod
    def agendar(paciente_id, medico_id, data_str, hora_str, observacao=''):
        """
        Valida e cria um agendamento.
        data_str: '25/12/2025' (formato brasileiro)
        hora_str: '14:30'
        """

        # Validação dos campos obrigatórios
        if not paciente_id:
            return False, 'Selecione um paciente!'
        if not medico_id:
            return False, 'Selecione um médico!'
        if not data_str:
            return False, 'Informe a data!'
        if not hora_str:
            return False, 'Informe o horário!'

        # Converte data e hora para o formato do MySQL
```



```
try:
    data_hora = datetime.strptime(
        f'{data_str} {hora_str}',
        '%d/%m/%Y %H:%M'
    )
except ValueError:
    return False, 'Data ou hora inválida! Use DD/MM/AAAA e HH:MM'

# Verifica se a consulta é no futuro
if data_hora <= datetime.now():
    return False, 'Não é possível agendar para datas passadas!'

# Verifica conflito de horário do médico
if AgendamentoModel.verificar_conflito(medico_id, data_hora):
    return False, 'Médico já tem consulta neste horário!'

try:
    AgendamentoModel.inserir(paciente_id, medico_id, data_hora,
observacao)

    return True, 'Consulta agendada com sucesso! ☒'
except Exception as e:
    return False, f'Erro ao agendar: {e}'

@staticmethod
def listar():
    return AgendamentoModel.listar_todos()
```

DICA DO PROFESSOR

O Controller retorna sempre uma TUPLA: (sucesso, mensagem)

Ex: return True, 'Cadastrado com sucesso!'

Ex: return False, 'Nome é obrigatório!'

Na View você faz: ok, msg = controller.salvar(...) e trata o resultado.

1.6 Exercícios da Aula 1

EXERCÍCIO PRÁTICO

EXERCÍCIO 1 – Criar o banco de dados

Abra o MySQL Workbench, execute o SQL da seção 1.3 e confirme que as tabelas foram criadas.

EXERCÍCIO 2 – Testar a conexão

Crie o arquivo database/conexao.py conforme o modelo, abra o terminal e execute:

python database/conexao.py

Deve aparecer: ☒ Conexão com MySQL OK!

EXERCÍCIO 3 – Criar o PlanoController

Com base no PacienteController, crie o controller/plano_controller.py

Ele deve ter os métodos: salvar(nome, codigo) e listar()

Validações: nome e código são obrigatórios.

EXERCÍCIO 4 – Criar o MedicoController

Crie controller/medico_controller.py com salvar(nome, especialidade, crm) e listar()

Validação extra: CRM deve ter pelo menos 5 caracteres.

Aula 2 – Interface Profissional com ttkbootstrap

2.1 O que é ttkbootstrap?

O Tkinter padrão cria telas funcionais, mas com visual dos anos 90. O **ttkbootstrap** é uma biblioteca que dá um 'banho de loja' no Tkinter, usando temas inspirados no Bootstrap (famoso framework web).

Tkinter Padrão	ttkbootstrap
Visual antigo e sem estilo	Visual moderno e profissional
Botões cinza e sem cor	Botões coloridos (success, danger...)
Só um tema (cinza padrão)	Mais de 20 temas prontos
<code>import tkinter as tk</code>	<code>import ttkbootstrap as tb</code>

2.2 Temas e Widgets Estilizados

Os temas disponíveis

O ttkbootstrap tem muitos temas. Os mais usados em sistemas profissionais são:

Tema	Estilo	Indicado para
flatly	Claro, minimalista	Sistemas médicos, empresariais
cosmo	Claro, moderno	Apps gerais, amigável
darkly	Escuro elegante	Sistemas noturnos, tech
superhero	Escuro azulado	Dashboards, relatórios
litera	Claro, tipografia boa	Documentos, relatórios

No MediCare+ usaremos o tema **'flatly'** – limpo, profissional e ideal para sistemas de saúde.

Como usar temas – Exemplo básico

CÓDIGO

```
# Importação principal - substitui o 'import tkinter as tk'
import ttkbootstrap as tb
```

```
from ttkbootstrap.constants import * # importa as constantes de estilo

# Cria a janela com um tema
app = tk.Window(themename='flatly')
app.title('MediCare+ - Agendamento Médico')
app.geometry('900x600')

# — WIDGETS ESTILIZADOS —

# Label simples
tk.Label(app, text='Bem-vindo ao MediCare+', font=('Arial', 16,
'bold')).pack(pady=20)

# Botões com cores (bootstyle)
tk.Button(app, text='Novo Agendamento', bootstyle='success').pack(pady=5)
tk.Button(app, text='Cancelar', bootstyle='danger').pack(pady=5)
tk.Button(app, text='Ver Relatório', bootstyle='info').pack(pady=5)
tk.Button(app, text='Atenção', bootstyle='warning').pack(pady=5)

app.mainloop()
```

Referência dos bootstyles

bootstyle=	Cor	Quando usar no MediCare+
'success'	 Verde	Confirmar, Salvar, Agendar
'danger'	 Vermelho	Cancelar, Excluir, Fechar
'info'	 Azul claro	Ver detalhes, Pesquisar
'warning'	 Amarelo	Atenção, pendências
'primary'	 Azul escuro	Ações principais
'secondary'	 Cinza	Ações secundárias
'light'	 Branco	Botões sutis

Widgets especiais do ttkbootstrap

O ttkbootstrap adiciona widgets que não existem no Tkinter padrão:

CÓDIGO

```
import ttkbootstrap as tk
```

```
from ttkbootstrap.constants import *

app = tb.Window(themename='flatly')

# — DateEntry - Seletor de data com calendário —
from ttkbootstrap.widgets import DateEntry
data = DateEntry(app, dateformat='%d/%m/%Y', bootstyle='info')
data.pack(pady=10)

# — Meter - Medidor circular (ótimo para dashboards) —
meter = tb.Meter(
    app,
    metersize=180,
    padding=5,
    amountused=75,      # valor atual
    metertype='semi',   # semicírculo
    subtext='Ocupação', # texto abaixo
    interactive=False,
    bootstyle='success'
)
meter.pack(pady=10)

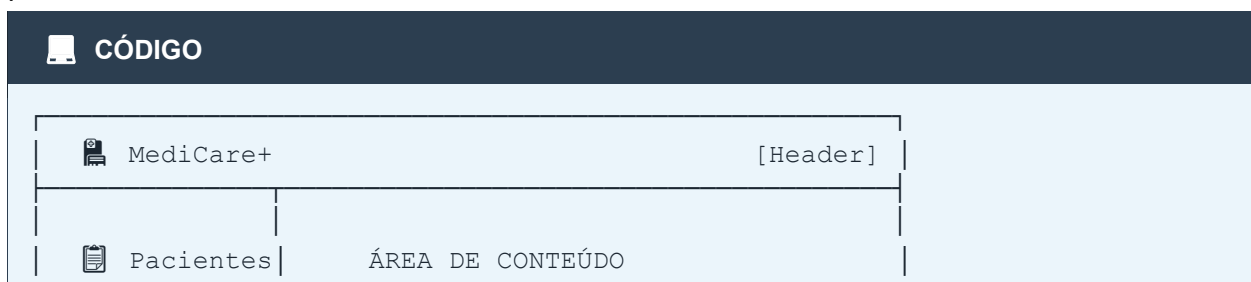
# — Floodgauge - Barra de progresso animada —
gauge = tb.Floodgauge(
    app,
    bootstyle='info',
    font=('Arial', 14),
    mask='Carregando {}%',
    value=0
)
gauge.pack(fill='x', padx=20, pady=10)

# — Scrolled Frame - Frame com barra de rolagem —
from ttkbootstrap.scrolled import ScrolledFrame
frame_scroll = ScrolledFrame(app, autohide=True)
frame_scroll.pack(fill='both', expand=True)

app.mainloop()
```

2.3 Criando o Menu Lateral (Dashboard)

O MediCare+ usa uma estrutura de Dashboard com menu lateral. É o layout mais usado em sistemas profissionais:



	(muda conforme o menu clicado)	
 Médicos		
 Planos		
 Agendar		
 Sair		
[Sidebar]		

Código do Dashboard Principal (view/main_view.py)

CÓDIGO

```
# view/main_view.py
import ttkbootstrap as tb
from ttkbootstrap.constants import *

class MainView:
    def __init__(self):
        # Cria a janela principal com tema médico
        self.app = tb.Window(themename='flatly')
        self.app.title('🏥 MediCare+ - Sistema de Agendamento')
        self.app.geometry('1100x680')
        self.app.resizable(True, True)

        self._criar_header()
        self._criar_layout()
        self._criar_sidebar()
        self._criar_area_conteudo()

        # Mostra tela inicial ao abrir
        self.mostrar_inicio()

    def _criar_header(self):
        """Barra superior com nome do sistema."""
        header = tb.Frame(self.app, bootstyle='primary', padding=(10, 8))
        header.pack(fill='x', side='top')

        tb.Label(
            header,
            text='🏥 MediCare+ - Sistema de Agendamento de Consultas',
            font=('Arial', 14, 'bold'),
            bootstyle='inverse-primary' # texto branco no fundo azul
        ).pack(side='left')

        # Mostra data/hora no canto direito
        self.label_hora = tb.Label(
            header,
            text='',
```

```
        font=('Arial', 11),
        bootstyle='inverse-primary'
    )
    self.label_hora.pack(side='right', padx=10)
    self._atualizar_hora() # inicia o relógio

def _atualizar_hora(self):
    """Atualiza o relógio a cada segundo."""
    from datetime import datetime
    agora = datetime.now().strftime('%d/%m/%Y %H:%M:%S')
    self.label_hora.config(text=agora)
    # Agenda nova atualização em 1 segundo (1000ms)
    self.app.after(1000, self._atualizar_hora)

def _criar_layout(self):
    """Frame principal que divide sidebar e conteúdo."""
    self.frame_main = tb.Frame(self.app)
    self.frame_main.pack(fill='both', expand=True)

def _criar_sidebar(self):
    """Menu lateral com botões de navegação."""
    sidebar = tb.Frame(
        self.frame_main,
        bootstyle='light',
        width=200,
        padding=(5, 10)
    )
    sidebar.pack(side='left', fill='y')
    sidebar.pack_propagate(False) # impede que o frame encolha

    # Logo/título do menu
    tb.Label(
        sidebar,
        text='MENU',
        font=('Arial', 10, 'bold'),
        foreground='gray'
    ).pack(pady=(10, 5))

    tb.Separator(sidebar).pack(fill='x', pady=5)

    # Definição dos botões do menu
    # (texto, ícone, função a chamar)
    menu_items = [
        ('🏠 Início', self.mostrar_inicio),
        ('👤 Pacientes', self.mostrar_pacientes),
        ('👨 Médicos', self.mostrar_medicos),
        ('👓 Planos', self.mostrar_planos),
        ('📅 Agendamentos', self.mostrar_agendamentos),
    ]

    # Cria cada botão do menu
    self.botoes_menu = []
```

```
for texto, comando in menu_items:
    btn = tb.Button(
        sidebar,
        text=texto,
        bootstyle='outline-primary', # borda azul, fundo branco
        width=20,
        command=comando
    )
    btn.pack(pady=3, padx=5, fill='x')
    self.botoes_menu.append(btn)

# Separador e botão de sair
tb.Separator(sidebar).pack(fill='x', pady=15)
tb.Button(
    sidebar,
    text='🏠 Sair',
    bootstyle='danger-outline',
    width=20,
    command=self.app.quit
).pack(pady=3, padx=5, fill='x')

def _criar_area_conteudo(self):
    """Área onde as telas aparecem."""
    self.frame_conteudo = tb.Frame(
        self.frame_main,
        bootstyle='default',
        padding=20
    )
    self.frame_conteudo.pack(side='left', fill='both', expand=True)

def _limpar_conteudo(self):
    """Remove todos os widgets da área de conteúdo."""
    for widget in self.frame_conteudo.winfo_children():
        widget.destroy()

# — FUNÇÕES DE NAVEGAÇÃO —————
def mostrar_inicio(self):
    self._limpar_conteudo()
    # Importa e exibe a tela de início
    from view.inicio_view import InicioView
    InicioView(self.frame_conteudo)

def mostrar_pacientes(self):
    self._limpar_conteudo()
    from view.paciente_view import PacienteView
    PacienteView(self.frame_conteudo)

def mostrar_medicos(self):
    self._limpar_conteudo()
    from view.medico_view import MedicoView
    MedicoView(self.frame_conteudo)

def mostrar_planos(self):
```



```
self._limpar_conteudo()
from view.plano_view import PlanoView
PlanoView(self.frame_conteudo)

def mostrar_agendamentos(self):
    self._limpar_conteudo()
    from view.agendamento_view import AgendamentoView
    AgendamentoView(self.frame_conteudo)

def iniciar(self):
    self.app.mainloop()
```

DICA DO PROFESSOR

O truque do Dashboard: a área de conteúdo é um frame vazio.
Quando clicamos no menu, chamamos `_limpar_conteudo()` e depois criamos novos widgets dentro de `self.frame_conteudo`.
É como trocar o 'slide' de uma apresentação!

2.4 Formulários Bonitos e Responsivos

Padrão de formulário usado no MediCare+

Veja abaixo o padrão que usamos em TODOS os formulários – é consistente e fácil de aprender:

CÓDIGO

```
# Estrutura padrão de formulário no MediCare+
import ttkbootstrap as tb
from ttkbootstrap.constants import *

def criar_formulario_padrao(parent):
    """Cria um formulário com o padrão visual do MediCare+."""

    # — TÍTULO DA SEÇÃO —————
    tb.Label(
        parent,
        text='📄 Cadastro de Pacientes',
        font=('Arial', 16, 'bold')
    ).pack(anchor='w', pady=(0, 5))

    tb.Separator(parent, bootstyle='primary').pack(fill='x', pady=(0, 20))

    # — FRAME DO FORMULÁRIO —————
    frame_form = tb.LabelFrame(
        parent,
        text='Dados do Paciente ',
        bootstyle='primary',
        padding=20
```

```
)
frame_form.pack(fill='x', pady=10)

# — CAMPOS EM GRADE (2 colunas) —————
# Linha 0: Nome e CPF
tb.Label(frame_form, text='Nome Completo *').grid(
    row=0, column=0, sticky='w', padx=5, pady=5
)
entry_nome = tb.Entry(frame_form, width=35, bootstyle='primary')
entry_nome.grid(row=1, column=0, padx=5, pady=5, sticky='ew')

tb.Label(frame_form, text='CPF *').grid(
    row=0, column=1, sticky='w', padx=5, pady=5
)
entry_cpf = tb.Entry(frame_form, width=20, bootstyle='primary')
entry_cpf.grid(row=1, column=1, padx=5, pady=5, sticky='ew')

# Linha 2: Telefone e Email
tb.Label(frame_form, text='Telefone').grid(
    row=2, column=0, sticky='w', padx=5, pady=5
)
entry_tel = tb.Entry(frame_form, width=20, bootstyle='info')
entry_tel.grid(row=3, column=0, padx=5, pady=5, sticky='ew')

tb.Label(frame_form, text='E-mail').grid(
    row=2, column=1, sticky='w', padx=5, pady=5
)
entry_email = tb.Entry(frame_form, width=35, bootstyle='info')
entry_email.grid(row=3, column=1, padx=5, pady=5, sticky='ew')

# Configurar expansão das colunas
frame_form.columnconfigure(0, weight=1)
frame_form.columnconfigure(1, weight=1)

# — BOTÕES DE AÇÃO —————
frame_botoes = tb.Frame(parent)
frame_botoes.pack(fill='x', pady=10)

tb.Button(
    frame_botoes,
    text='☑ Salvar',
    bootstyle='success',
    width=15
).pack(side='left', padx=5)

tb.Button(
    frame_botoes,
    text='🧼 Limpar',
    bootstyle='secondary',
    width=15
).pack(side='left', padx=5)

tb.Button(
```

```
frame_botoes,
text='✕ Cancelar',
bootstyle='danger-outline',
width=15
).pack(side='right', padx=5)
```

Treeview – Tabela de Dados

O **Treeview** é o widget que exibe dados em forma de tabela. Usamos em todas as telas de listagem:

CÓDIGO

```
# Como criar e popular uma tabela Treeview com ttkbootstrap
from ttkbootstrap.tableview import Tableview

def criar_tabela_pacientes(parent, dados):
    """
    dados: lista de dicionários vindos do banco
    Exemplo: [{'id': 1, 'nome': 'João', 'cpf': '123', ...}, ...]
    """

    # Definir as colunas
    colunas = [
        {'text': 'ID',          'stretch': False, 'width': 50},
        {'text': 'Nome',       'stretch': True,  'width': 200},
        {'text': 'CPF',        'stretch': False, 'width': 120},
        {'text': 'Telefone',   'stretch': False, 'width': 120},
        {'text': 'Plano',      'stretch': True,  'width': 150},
    ]

    # Preparar as linhas
    linhas = []
    for p in dados:
        linhas.append((
            p['id'],
            p['nome'],
            p['cpf'],
            p.get('telefone', ''),
            p.get('nome_plano', 'Particular')
        ))

    # Criar o Tableview (é o Treeview turbinado do ttkbootstrap)
    tabela = Tableview(
        master=parent,
        coldata=colunas,
        rowdata=linhas,
        paginated=True,      # adiciona paginação automática!
        searchable=True,     # adiciona campo de busca!
        bootstyle='primary',
        stripecolor=('f0f8ff', None) # linhas alternadas
    )
    tabela.pack(fill='both', expand=True, pady=10)
    return tabela
```

2.5 Exercícios da Aula 2

EXERCÍCIO PRÁTICO

EXERCÍCIO 1 – Primeiro tema

Crie um arquivo teste_tema.py e teste pelo menos 3 temas diferentes.

Tente: flatly, darkly e superhero. Qual você prefere?

EXERCÍCIO 2 – Botões coloridos

Crie uma janela com 6 botões, um para cada bootstyle.

Quando clicar em cada botão, mostre uma messagebox com o nome do estilo.

EXERCÍCIO 3 – Layout duas colunas

Recrie o formulário padrão de Pacientes só com a interface (sem banco).

Use o grid() com 2 colunas: Nome + CPF na primeira linha, Telefone + Email na segunda.

EXERCÍCIO 4 – Menu lateral

Crie um mini-dashboard com sidebar de 3 botões.

Ao clicar em cada botão, troque um Label central com o nome da tela.

Aula 3 – Projeto Final Completo: MediCare+

Chegou a hora de montar tudo! Vamos unir MVC + MySQL + ttkbootstrap num sistema completo e funcional.

PROJETO MediCare+

Nesta aula você vai criar todas as telas do MediCare+:

- Tela de Login (com verificação de senha)
- Cadastro de Pacientes (com listagem e edição)
- Cadastro de Médicos
- Cadastro de Planos de Saúde
- Agendamento de Consultas (com calendário e validações)
- Listagem de todos os agendamentos

3.1 Tela de Login (view/login_view.py)

Vamos começar pelo login, que é a porta de entrada do sistema:

CÓDIGO

```
# view/login_view.py
import ttkbootstrap as tb
from ttkbootstrap.constants import *
from ttkbootstrap import dialogs

class LoginView:
    def __init__(self):
        self.app = tb.Window(themename='flatly')
        self.app.title('MediCare+ - Login')
        self.app.geometry('420x520')
        self.app.resizable(False, False)
        self._centralizar_janela(420, 520)
        self._criar_tela()

    def _centralizar_janela(self, w, h):
        """Coloca a janela no centro da tela."""
        screen_w = self.app.winfo_screenwidth()
        screen_h = self.app.winfo_screenheight()
        x = (screen_w // 2) - (w // 2)
        y = (screen_h // 2) - (h // 2)
        self.app.geometry(f'{w}x{h}+{x}+{y}')

    def _criar_tela(self):
        # — CABEÇALHO AZUL —
        header = tb.Frame(self.app, bootstyle='primary', padding=30)
        header.pack(fill='x')
```

```
tb.Label(
    header,
    text='🏠',
    font=('Arial', 40),
    bootstyle='inverse-primary'
).pack()

tb.Label(
    header,
    text='MediCare+',
    font=('Arial', 22, 'bold'),
    bootstyle='inverse-primary'
).pack()

tb.Label(
    header,
    text='Sistema de Agendamento Médico',
    font=('Arial', 10),
    bootstyle='inverse-primary'
).pack()

# — FORMULÁRIO DE LOGIN —————
frame_form = tb.Frame(self.app, padding=30)
frame_form.pack(fill='both', expand=True)

11) tb.Label(frame_form, text='Usuário', font=('Arial',
).pack(anchor='w', pady=(10,2))
self.entry_usuario = tb.Entry(
    frame_form, width=30, bootstyle='primary', font=('Arial', 12)
)
self.entry_usuario.pack(fill='x', ipady=5)
self.entry_usuario.insert(0, 'admin') # valor padrão para teste

tb.Label(frame_form, text='Senha', font=('Arial',
11)).pack(anchor='w', pady=(15,2))
self.entry_senha = tb.Entry(
    frame_form, width=30, bootstyle='primary',
    font=('Arial', 12), show='●' # oculta a senha
)
self.entry_senha.pack(fill='x', ipady=5)
self.entry_senha.insert(0, '1234')

# Bind Enter para fazer login
self.entry_senha.bind('<Return>', lambda e: self._fazer_login())

tb.Button(
    frame_form,
    text='🔑 Entrar',
    bootstyle='primary',
    width=20,
    command=self._fazer_login
).pack(pady=20, ipady=8)
```

```
def _fazer_login(self):
    usuario = self.entry_usuario.get()
    senha = self.entry_senha.get()

    # Validação simples (em sistema real, verificar no banco!)
    if usuario == 'admin' and senha == '1234':
        self.app.destroy() # fecha a tela de login
        from view.main_view import MainView
        sistema = MainView()
        sistema.iniciar() # abre o sistema principal
    else:
        dialogs.Messagebox.show_error(
            'Usuário ou senha incorretos!',
            'Erro de Login'
        )
        self.entry_senha.delete(0, 'end') # limpa a senha

def iniciar(self):
    self.app.mainloop()
```

3.2 Cadastro de Pacientes (view/paciente_view.py)

CÓDIGO

```
# view/paciente_view.py
import ttkbootstrap as tb
from ttkbootstrap.constants import *
from ttkbootstrap import dialogs
from ttkbootstrap.tableview import Tableview
from controller.paciente_controller import PacienteController
from controller.plano_controller import PlanoController

class PacienteView:
    def __init__(self, parent):
        self.parent = parent
        self.id_selecionado = None # guarda ID ao selecionar na tabela
        self._criar_tela()

    def _criar_tela(self):
        # — TÍTULO —
        tb.Label(
            self.parent,
            text='👤 Cadastro de Pacientes',
            font=('Arial', 16, 'bold')
        ).pack(anchor='w', pady=(0, 5))
        tb.Separator(self.parent, bootstyle='primary').pack(fill='x',
            pady=(0, 15))

        # — NOTEBOOK (abas: Cadastrar | Listar) —
        notebook = tb.Notebook(self.parent, bootstyle='primary')
        notebook.pack(fill='both', expand=True)
```

```
# Aba 1: Formulário de cadastro
aba_form = tb.Frame(notebook, padding=15)
notebook.add(aba_form, text=' + Novo Paciente ')
self._criar_formulario(aba_form)

# Aba 2: Listagem
aba_lista = tb.Frame(notebook, padding=15)
notebook.add(aba_lista, text=' 📋 Lista de Pacientes ')
self._criar_listagem(aba_lista)

def _criar_formulario(self, parent):
    """Formulário de cadastro de paciente."""
    # Frame principal do formulário
    frame = tb.LabelFrame(
        parent, text=' Dados do Paciente ', bootstyle='primary',
padding=20
    )
    frame.pack(fill='x', pady=10)

    # — Linha 1: Nome (ocupa toda a largura) —
    tb.Label(frame, text='Nome Completo *', font=('Arial', 10,
'bold')).grid(
        row=0, column=0, columnspan=2, sticky='w', padx=5, pady=(5,2)
    )
    self.entry_nome = tb.Entry(frame, width=60, bootstyle='primary',
font=('Arial',11))
    self.entry_nome.grid(row=1, column=0, columnspan=2, padx=5, pady=5,
sticky='ew')

    # — Linha 2: CPF e Telefone —
    tb.Label(frame, text='CPF *', font=('Arial', 10, 'bold')).grid(
        row=2, column=0, sticky='w', padx=5, pady=(10,2)
    )
    self.entry_cpf = tb.Entry(frame, width=20, bootstyle='primary',
font=('Arial',11))
    self.entry_cpf.grid(row=3, column=0, padx=5, pady=5, sticky='ew')

    tb.Label(frame, text='Telefone', font=('Arial', 10, 'bold')).grid(
        row=2, column=1, sticky='w', padx=5, pady=(10,2)
    )
    self.entry_tel = tb.Entry(frame, width=20, bootstyle='info',
font=('Arial',11))
    self.entry_tel.grid(row=3, column=1, padx=5, pady=5, sticky='ew')

    # — Linha 3: Email e Plano —
    tb.Label(frame, text='E-mail', font=('Arial', 10, 'bold')).grid(
        row=4, column=0, sticky='w', padx=5, pady=(10,2)
    )
    self.entry_email = tb.Entry(frame, width=35, bootstyle='info',
font=('Arial',11))
    self.entry_email.grid(row=5, column=0, padx=5, pady=5, sticky='ew')
```



```
        tb.Label(frame, text='Plano de Saúde', font=('Arial', 10,
'bold')).grid(
            row=4, column=1, sticky='w', padx=5, pady=(10,2)
        )

        # Combobox para selecionar o plano
        self.combo_plano = tb.Combobox(
            frame, state='readonly', bootstyle='info', font=('Arial',11),
width=25
        )
        self.combo_plano.grid(row=5, column=1, padx=5, pady=5, sticky='ew')
        self._carregar_planos() # popula o combobox

        # Expande as colunas igualmente
        frame.columnconfigure(0, weight=1)
        frame.columnconfigure(1, weight=1)

        # — Botões —————
        frame_btn = tb.Frame(parent)
        frame_btn.pack(fill='x', pady=15)

        tb.Button(
            frame_btn, text='☑ Salvar Paciente', bootstyle='success',
            width=20, command=self._salvar
        ).pack(side='left', padx=5, ipady=5)

        tb.Button(
            frame_btn, text='🗑 Limpar Campos', bootstyle='secondary',
            width=18, command=self._limpar
        ).pack(side='left', padx=5, ipady=5)

    def _carregar_planos(self):
        """Popula o Combobox com os planos do banco."""
        self.planos_lista = PlanoController.listar()
        nomes = [p['nome'] for p in self.planos_lista]
        self.combo_plano['values'] = nomes
        if nomes:
            self.combo_plano.current(0) # seleciona o primeiro

    def _get_plano_id(self):
        """Retorna o ID do plano selecionado no combobox."""
        idx = self.combo_plano.current()
        if idx >= 0 and self.planos_lista:
            return self.planos_lista[idx]['id']
        return None

    def _salvar(self):
        """Coleta os dados e chama o controller para salvar."""
        nome = self.entry_nome.get().strip()
        cpf = self.entry_cpf.get().strip()
        telefone = self.entry_tel.get().strip()
        email = self.entry_email.get().strip()
        plano_id = self._get_plano_id()
```

```
# Chama o controller - ele faz a validação
ok, mensagem = PacienteController.salvar(nome, cpf, telefone,
email, plano_id)

if ok:
    dialogs.Messagebox.show_info(mensagem, 'Sucesso')
    self._limpar()
else:
    dialogs.Messagebox.show_error(mensagem, 'Erro')

def _limpar(self):
    """Limpa todos os campos do formulário."""
    self.entry_nome.delete(0, 'end')
    self.entry_cpf.delete(0, 'end')
    self.entry_tel.delete(0, 'end')
    self.entry_email.delete(0, 'end')
    self.entry_nome.focus() # foca no primeiro campo

def _criar_listagem(self, parent):
    """Aba de listagem de pacientes com tabela."""
    # Botão para atualizar a lista
    tb.Button(
        parent, text='🔄 Atualizar Lista', bootstyle='info-outline',
        command=self._atualizar_tabela
    ).pack(anchor='e', pady=(0,10))

    # Container da tabela
    self.frame_tabela = tb.Frame(parent)
    self.frame_tabela.pack(fill='both', expand=True)

    self._atualizar_tabela()

def _atualizar_tabela(self):
    """Busca dados no banco e atualiza a tabela."""
    # Limpa a tabela anterior
    for w in self.frame_tabela.winfo_children():
        w.destroy()

    # Busca os dados via controller
    dados = PacienteController.listar()

    # Define colunas
    colunas = [
        {'text': 'ID',          'stretch': False, 'width': 50},
        {'text': 'Nome',        'stretch': True,  'width': 200},
        {'text': 'CPF',         'stretch': False, 'width': 130},
        {'text': 'Telefone',    'stretch': False, 'width': 120},
        {'text': 'E-mail',      'stretch': True,  'width': 180},
        {'text': 'Plano',       'stretch': True,  'width': 130},
    ]

    # Prepara as linhas
```

```
linhas = [(
    p['id'], p['nome'], p['cpf'],
    p.get('telefone', ''), p.get('email', ''),
    p.get('nome_plano', 'Particular')
) for p in dados]

Tableview(
    master=self.frame_tabela,
    coldata=colunas,
    rowdata=linhas,
    paginated=True,
    searchable=True,
    bootstyle='primary',
    stripecolor=('f0f8ff', None)
).pack(fill='both', expand=True)
```

3.3 Cadastro de Médicos (view/medico_view.py)

CÓDIGO

```
# view/medico_view.py
import ttkbootstrap as tb
from ttkbootstrap.constants import *
from ttkbootstrap import dialogs
from ttkbootstrap.tableview import Tableview
from controller.medico_controller import MedicoController

ESPECIALIDADES = [
    'Clínico Geral', 'Cardiologia', 'Ortopedia', 'Neurologia',
    'Pediatria', 'Ginecologia', 'Dermatologia', 'Oftalmologia',
    'Psiquiatria', 'Endocrinologia', 'Urologia', 'Oncologia'
]

class MedicoView:
    def __init__(self, parent):
        self.parent = parent
        self._criar_tela()

    def _criar_tela(self):
        tb.Label(
            self.parent,
            text='👨‍⚕️ Cadastro de Médicos',
            font=('Arial', 16, 'bold')
        ).pack(anchor='w', pady=(0,5))
        tb.Separator(self.parent, bootstyle='success').pack(fill='x',
pady=(0,15))

        notebook = tb.Notebook(self.parent, bootstyle='success')
        notebook.pack(fill='both', expand=True)

        aba_form = tb.Frame(notebook, padding=15)
```

```
notebook.add(aba_form, text=' + Novo Médico ')
self._criar_formulario(aba_form)

aba_lista = tb.Frame(notebook, padding=15)
notebook.add(aba_lista, text=' 📋 Lista de Médicos ')
self._criar_listagem(aba_lista)

def _criar_formulario(self, parent):
    frame = tb.LabelFrame(
        parent, text=' Dados do Médico ', bootstyle='success',
padding=20
    )
    frame.pack(fill='x', pady=10)

    # Nome completo
    tb.Label(frame, text='Nome Completo *',
font=('Arial',10,'bold')).grid(
        row=0, column=0, columnspan=2, sticky='w', padx=5, pady=(5,2)
    )
    self.entry_nome = tb.Entry(frame, width=50, font=('Arial',11),
bootstyle='success')
    self.entry_nome.grid(row=1, column=0, columnspan=2, padx=5, pady=5,
sticky='ew')

    # Especialidade (Combobox) e CRM
    tb.Label(frame, text='Especialidade *',
font=('Arial',10,'bold')).grid(
        row=2, column=0, sticky='w', padx=5, pady=(10,2)
    )
    self.combo_esp = tb.Combobox(
        frame, values=ESPECIALIDADES, state='readonly',
bootstyle='success', width=28
    )
    self.combo_esp.grid(row=3, column=0, padx=5, pady=5, sticky='ew')
    self.combo_esp.current(0)

    tb.Label(frame, text='CRM *', font=('Arial',10,'bold')).grid(
        row=2, column=1, sticky='w', padx=5, pady=(10,2)
    )
    self.entry_crm = tb.Entry(frame, width=20, font=('Arial',11),
bootstyle='success')
    self.entry_crm.grid(row=3, column=1, padx=5, pady=5, sticky='ew')

    frame.columnconfigure(0, weight=1)
    frame.columnconfigure(1, weight=1)

    # Botões
    frame_btn = tb.Frame(parent)
    frame_btn.pack(fill='x', pady=15)

    tb.Button(
        frame_btn, text='📌 Salvar Médico', bootstyle='success',
        width=20, command=self._salvar
```

```
        ).pack(side='left', padx=5, ipady=5)

        tb.Button(
            frame_btn, text='🧼 Limpar', bootstyle='secondary',
            width=15, command=self._limpar
        ).pack(side='left', padx=5, ipady=5)

    def _salvar(self):
        nome = self.entry_nome.get().strip()
        esp = self.combo_esp.get()
        crm = self.entry_crm.get().strip()

        ok, msg = MedicoController.salvar(nome, esp, crm)
        if ok:
            dialogs.Messagebox.show_info(msg, 'Sucesso')
            self._limpar()
        else:
            dialogs.Messagebox.show_error(msg, 'Erro')

    def _limpar(self):
        self.entry_nome.delete(0, 'end')
        self.entry_crm.delete(0, 'end')
        self.combo_esp.current(0)
        self.entry_nome.focus()

    def _criar_listagem(self, parent):
        tb.Button(
            parent, text='🔄 Atualizar',
            bootstyle='success-outline', command=self._atualizar_tabela
        ).pack(anchor='e', pady=(0,10))
        self.frame_tab = tb.Frame(parent)
        self.frame_tab.pack(fill='both', expand=True)
        self._atualizar_tabela()

    def _atualizar_tabela(self):
        for w in self.frame_tab.winfo_children(): w.destroy()
        dados = MedicoController.listar()
        colunas = [
            {'text': 'ID', 'stretch': False, 'width': 50},
            {'text': 'Nome', 'stretch': True, 'width': 200},
            {'text': 'Especialidade', 'stretch': True, 'width': 160},
            {'text': 'CRM', 'stretch': False, 'width': 120},
        ]
        linhas = [(m['id'], m['nome'], m['especialidade'], m['crm']) for m
in dados]
        Tableview(
            master=self.frame_tab, coldata=colunas, rowdata=linhas,
            paginated=True, searchable=True, bootstyle='success',
            stripecolor=('f0fff0', None)
        ).pack(fill='both', expand=True)
```

3.4 Cadastro de Planos de Saúde (view/plano_view.py)

 CÓDIGO

```
# view/plano_view.py
import ttkbootstrap as tb
from ttkbootstrap.constants import *
from ttkbootstrap.dialogs import dialogs
from ttkbootstrap.tableview import Tableview
from controller.plano_controller import PlanoController

class PlanoView:
    def __init__(self, parent):
        self.parent = parent
        self._criar_tela()

    def _criar_tela(self):
        tb.Label(
            self.parent, text='🔑 Cadastro de Planos de Saúde',
            font=('Arial', 16, 'bold')
        ).pack(anchor='w', pady=(0,5))
        tb.Separator(self.parent, bootstyle='warning').pack(fill='x',
pady=(0,15))

        notebook = tb.Notebook(self.parent, bootstyle='warning')
        notebook.pack(fill='both', expand=True)

        aba_form = tb.Frame(notebook, padding=20)
        notebook.add(aba_form, text=' + Novo Plano ')

        frame = tb.LabelFrame(
            aba_form, text=' Dados do Plano ', bootstyle='warning',
padding=25
        )
        frame.pack(fill='x')

        tb.Label(frame, text='Nome do Plano *',
font=('Arial',10,'bold')).grid(
            row=0, column=0, sticky='w', padx=5, pady=(5,2)
        )
        self.entry_nome = tb.Entry(frame, width=35, font=('Arial',11),
bootstyle='warning')
        self.entry_nome.grid(row=1, column=0, padx=5, pady=5, sticky='ew')

        tb.Label(frame, text='Código *', font=('Arial',10,'bold')).grid(
            row=0, column=1, sticky='w', padx=5, pady=(5,2)
        )
        self.entry_cod = tb.Entry(frame, width=20, font=('Arial',11),
bootstyle='warning')
        self.entry_cod.grid(row=1, column=1, padx=5, pady=5, sticky='ew')

        frame.columnconfigure(0, weight=2)
        frame.columnconfigure(1, weight=1)

        tb.Button(
```

```
        aba_form, text='☑ Salvar Plano', bootstyle='warning',
        width=18, command=self._salvar
    ).pack(pady=15, ipady=5)

    aba_lista = tb.Frame(notebook, padding=15)
    notebook.add(aba_lista, text='📋 Lista de Planos ')
    self._criar_listagem(aba_lista)

def _salvar(self):
    nome = self.entry_nome.get().strip()
    cod = self.entry_cod.get().strip()
    ok, msg = PlanoController.salvar(nome, cod)
    if ok:
        dialogs.Messagebox.show_info(msg, 'Sucesso')
        self.entry_nome.delete(0, 'end')
        self.entry_cod.delete(0, 'end')
    else:
        dialogs.Messagebox.show_error(msg, 'Erro')

def _criar_listagem(self, parent):
    tb.Button(
        parent, text='🔄 Atualizar',
        bootstyle='warning-outline', command=self._atualizar_tabela
    ).pack(anchor='e', pady=(0,10))
    self.frame_tab = tb.Frame(parent)
    self.frame_tab.pack(fill='both', expand=True)
    self._atualizar_tabela()

def _atualizar_tabela(self):
    for w in self.frame_tab.winfo_children(): w.destroy()
    dados = PlanoController.listar()
    colunas = [
        {'text': 'ID', 'stretch': False, 'width': 60},
        {'text': 'Nome', 'stretch': True, 'width': 250},
        {'text': 'Código', 'stretch': False, 'width': 120},
    ]
    linhas = [(p['id'], p['nome'], p['codigo']) for p in dados]
    Tableview(
        master=self.frame_tab, coldata=colunas, rowdata=linhas,
        paginated=True, searchable=True, bootstyle='warning',
    ).pack(fill='both', expand=True)
```

3.5 Tela de Agendamento (view/agendamento_view.py)

Esta é a tela mais importante do sistema – onde tudo se conecta:

CÓDIGO

```
# view/agendamento_view.py
import ttkbootstrap as tb
from ttkbootstrap.constants import *
from ttkbootstrap import dialogs
```

```
from ttkbootstrap.tableview import Tableview
from ttkbootstrap.widgets import DateEntry
from controller.agendamento_controller import AgendamentoController
from controller.paciente_controller import PacienteController
from controller.medico_controller import MedicoController

class AgendamentoView:
    def __init__(self, parent):
        self.parent = parent
        self.pacientes = [] # lista de pacientes do banco
        self.medicos = [] # lista de médicos do banco
        self._criar_tela()

    def _criar_tela(self):
        tb.Label(
            self.parent, text='📅 Agendamento de Consultas',
            font=('Arial', 16, 'bold')
        ).pack(anchor='w', pady=(0,5))
        tb.Separator(self.parent, bootstyle='info').pack(fill='x',
pady=(0,15))

        notebook = tb.Notebook(self.parent, bootstyle='info')
        notebook.pack(fill='both', expand=True)

        aba_form = tb.Frame(notebook, padding=15)
        notebook.add(aba_form, text='📅 Nova Consulta ')
        self._criar_formulario(aba_form)

        aba_lista = tb.Frame(notebook, padding=15)
        notebook.add(aba_lista, text='📋 Todos os Agendamentos ')
        self._criar_listagem(aba_lista)

    def _criar_formulario(self, parent):
        # — Frame: Selecionar Paciente —————
        frame_pac = tb.LabelFrame(
            parent, text='👤 Paciente ', bootstyle='primary', padding=15
        )
        frame_pac.pack(fill='x', pady=5)

        tb.Label(frame_pac, text='Selecione o Paciente *',
font=('Arial',10,'bold')).pack(anchor='w')
        self.combo_pac = tb.Combobox(
            frame_pac, state='readonly', bootstyle='primary',
font=('Arial',11), width=50
        )
        self.combo_pac.pack(fill='x', pady=5)
        self._carregar_pacientes()

        # — Frame: Selecionar Médico —————
        frame_med = tb.LabelFrame(
            parent, text='👨 Médico ', bootstyle='success', padding=15
        )
        frame_med.pack(fill='x', pady=5)
```



```
        tb.Label(frame_med, text='Selecione o Médico *',
font=('Arial',10,'bold')).pack(anchor='w')
        self.combo_med = tb.Combobox(
            frame_med, state='readonly', bootstyle='success',
font=('Arial',11), width=50
        )
        self.combo_med.pack(fill='x', pady=5)
        self._carregar_medicos()

# ——— Frame: Data e Horário ———
frame_dt = tb.LabelFrame(
    parent, text='📅 Data e Horário ', bootstyle='info',
padding=15
)
frame_dt.pack(fill='x', pady=5)

# Data com calendário (DateEntry)
tb.Label(frame_dt, text='Data da Consulta *',
font=('Arial',10,'bold')).grid(
    row=0, column=0, sticky='w', padx=5, pady=(5,2)
)
self.date_entry = DateEntry(
    frame_dt,
    dateformat='%d/%m/%Y',
    bootstyle='info',
    width=20
)
self.date_entry.grid(row=1, column=0, padx=5, pady=5, sticky='ew')

# Horário com Combobox (horários disponíveis)
tb.Label(frame_dt, text='Horário *',
font=('Arial',10,'bold')).grid(
    row=0, column=1, sticky='w', padx=5, pady=(5,2)
)
horarios = [f'{h:02d}:{m:02d}'
            for h in range(7, 19)    # das 07h às 18h
            for m in [0, 30]]        # a cada 30 minutos
self.combo_hora = tb.Combobox(
    frame_dt, values=horarios, state='readonly',
    bootstyle='info', width=10
)
self.combo_hora.grid(row=1, column=1, padx=5, pady=5, sticky='ew')
self.combo_hora.current(4) # padrão: 09:00

frame_dt.columnconfigure(0, weight=2)
frame_dt.columnconfigure(1, weight=1)

# Observação
tb.Label(frame_dt, text='Observações',
font=('Arial',10,'bold')).grid(
    row=2, column=0, columnspan=2, sticky='w', padx=5, pady=(10,2)
)
self.text_obs = tb.Text(
```

```
        frame_dt, height=3, width=60, font=('Arial',10)
    )
    self.text_obs.grid(row=3, column=0, columnspan=2, padx=5, pady=5,
sticky='ew')

    # — Botão Agendar —————
    tb.Button(
        parent,
        text='☑ CONFIRMAR AGENDAMENTO',
        bootstyle='success',
        width=30,
        command=self._agendar
    ).pack(pady=20, ipady=8)

def _carregar_pacientes(self):
    """Popula o combobox com os pacientes do banco."""
    self.pacientes = PacienteController.listar()
    nomes = [f"{p['id']} - {p['nome']}" for p in self.pacientes]
    self.combo_pac['values'] = nomes
    if nomes: self.combo_pac.current(0)

def _carregar_medicos(self):
    """Popula o combobox com os médicos do banco."""
    self.medicos = MedicoController.listar()
    nomes = [f"{m['nome']} - {m['especialidade']}" for m in
self.medicos]
    self.combo_med['values'] = nomes
    if nomes: self.combo_med.current(0)

def _agendar(self):
    """Coleta dados e chama o controller para agendar."""
    # Pega IDs selecionados nos comboboxes
    idx_pac = self.combo_pac.current()
    idx_med = self.combo_med.current()

    if idx_pac < 0 or not self.pacientes:
        dialogs.Messagebox.show_error('Selecione um paciente!',
'Atenção')
        return
    if idx_med < 0 or not self.medicos:
        dialogs.Messagebox.show_error('Selecione um médico!',
'Atenção')
        return

    paciente_id = self.pacientes[idx_pac]['id']
    medico_id = self.medicos[idx_med]['id']

    # Pega data e hora selecionadas
    data_str = self.date_entry.entry.get()
    hora_str = self.combo_hora.get()
    obs = self.text_obs.get('1.0', 'end').strip()

    # Chama o controller - ele valida e salva
```

```
ok, msg = AgendamentoController.agendar(
    paciente_id, medico_id, data_str, hora_str, obs
)

if ok:
    dialogs.Messagebox.show_info(msg, '📅 Agendamento Confirmado')
    self.text_obs.delete('1.0', 'end')
else:
    dialogs.Messagebox.show_error(msg, '❌ Erro no Agendamento')

def _criar_listagem(self, parent):
    """Aba com todos os agendamentos."""
    frame_topo = tb.Frame(parent)
    frame_topo.pack(fill='x', pady=(0,10))

    tb.Label(frame_topo, text='Todos os Agendamentos',
font=('Arial',13,'bold')).pack(side='left')
    tb.Button(
        frame_topo, text='🔄 Atualizar',
        bootstyle='info-outline', command=self._atualizar_tabela
    ).pack(side='right')

    self.frame_tab = tb.Frame(parent)
    self.frame_tab.pack(fill='both', expand=True)
    self._atualizar_tabela()

def _atualizar_tabela(self):
    for w in self.frame_tab.winfo_children(): w.destroy()
    dados = AgendamentoController.listar()
    colunas = [
        {'text': 'ID', 'stretch': False, 'width': 50},
        {'text': 'Paciente', 'stretch': True, 'width': 180},
        {'text': 'Médico', 'stretch': True, 'width': 180},
        {'text': 'Especialidade', 'stretch': True, 'width': 140},
        {'text': 'Data/Hora', 'stretch': False, 'width': 150},
        {'text': 'Status', 'stretch': False, 'width': 100},
    ]
    linhas = [(
        a['id'], a['paciente'], a['medico'],
        a['especialidade'],
        str(a['data_hora']).replace('T', ' ')[:16],
        a['status'].upper()
    ) for a in dados]
    Tableview(
        master=self.frame_tab, coldata=colunas, rowdata=linhas,
        paginated=True, searchable=True, bootstyle='info',
        stripecolor=('e8f4f8', None)
    ).pack(fill='both', expand=True)
```

3.6 Controllers Complementares

controller/medico_controller.py

 CÓDIGO

```
# controller/medico_controller.py
from model.medico_model import MedicoModel

class MedicoController:

    @staticmethod
    def salvar(nome, especialidade, crm):
        if not nome.strip():
            return False, 'Nome do médico é obrigatório!'
        if not especialidade:
            return False, 'Especialidade é obrigatória!'
        if not crm.strip() or len(crm.strip()) < 5:
            return False, 'CRM inválido! Mínimo 5 caracteres.'
        try:
            MedicoModel.inserir(nome, especialidade, crm)
            return True, f'Médico {nome} cadastrado com sucesso!'
        except ValueError as e:
            return False, str(e)
        except Exception as e:
            return False, f'Erro: {e}'

    @staticmethod
    def listar():
        return MedicoModel.listar_todos()
```

controller/plano_controller.py

 CÓDIGO

```
# controller/plano_controller.py
from model.plano_model import PlanoModel

class PlanoController:

    @staticmethod
    def salvar(nome, codigo):
        if not nome.strip():
            return False, 'Nome do plano é obrigatório!'
        if not codigo.strip():
            return False, 'Código do plano é obrigatório!'
        try:
            PlanoModel.inserir(nome, codigo)
            return True, f'Plano "{nome}" cadastrado com sucesso!'
        except Exception as e:
            return False, f'Erro: {e}'

    @staticmethod
    def listar():
        return PlanoModel.listar_todos()
```

3.7 Arquivo Principal (main.py)

Este é o único arquivo que você deve executar:

CÓDIGO

```
# main.py ← EXECUTE ESTE ARQUIVO!
# Forma de executar: python main.py

from view.login_view import LoginView

if __name__ == '__main__':
    # Ponto de entrada do sistema
    app = LoginView()
    app.iniciar()
```

Como executar o MediCare+

1. Abra o terminal na pasta raiz do projeto (onde está o main.py)
 2. Execute: `python main.py`
 3. Login padrão: usuário = admin | senha = 1234
- NUNCA execute: `python view/main_view.py` (causa erro de importação!)

3.8 Tela de Início (view/inicio_view.py)

CÓDIGO

```
# view/inicio_view.py
import ttkbootstrap as tb
from ttkbootstrap.constants import *
from datetime import datetime
from controller.paciente_controller import PacienteController
from controller.medico_controller import MedicoController
from controller.agendamento_controller import AgendamentoController

class InicioView:
    def __init__(self, parent):
        self.parent = parent
        self._criar_dashboard()

    def _criar_dashboard(self):
        """Painel inicial com resumo do sistema."""
        # Saudação
        hora = datetime.now().hour
        if hora < 12:    saudacao = 'Bom dia'
        elif hora < 18: saudacao = 'Boa tarde'
        else:           saudacao = 'Boa noite'
```

```
tb.Label(
    self.parent,
    text=f'{saudacao}! Bem-vindo ao MediCare+',
    font=('Arial', 18, 'bold')
).pack(anchor='w', pady=(0,5))
tb.Separator(self.parent, bootstyle='primary').pack(fill='x',
pady=(0,25))

# — Cards de Resumo —
frame_cards = tb.Frame(self.parent)
frame_cards.pack(fill='x', pady=10)

# Busca totais do banco
total_pac = len(PacienteController.listar())
total_med = len(MedicoController.listar())
total_age = len(AgendamentoController.listar())

# Função para criar cada card
def card(parent, icone, titulo, valor, cor):
    f = tb.Frame(parent, bootstyle=cor, padding=20)
    f.pack(side='left', fill='both', expand=True, padx=8)
    tb.Label(f, text=icone, font=('Arial',30), bootstyle=f'inverse-
{cor}').pack()
    tb.Label(f, text=str(valor), font=('Arial',32,'bold'),
bootstyle=f'inverse-{cor}').pack()
    tb.Label(f, text=titulo, font=('Arial',12),
bootstyle=f'inverse-{cor}').pack()

card(frame_cards, '👤', 'Pacientes', total_pac, 'primary')
card(frame_cards, '👨‍⚕️', 'Médicos', total_med, 'success')
card(frame_cards, '📅', 'Agendamentos', total_age, 'info')

# Dica do dia
tb.Label(
    self.parent,
    text='💡 Use o menu lateral para navegar entre as telas.',
    font=('Arial', 11),
    foreground='gray'
).pack(anchor='w', pady=30)
```

3.9 Executando o Sistema Completo

EXERCÍCIO PRÁTICO

EXERCÍCIO FINAL – Montar e executar o MediCare+

Passo 1: Crie toda a estrutura de pastas e arquivos `__init__.py`

Passo 2: Execute o SQL do banco de dados no MySQL

Passo 3: Configure o arquivo `database/conexao.py` com sua senha



Passo 4: Crie todos os arquivos de model, controller e view

Passo 5: Execute: `python main.py`

Passo 6: Faça login com admin/1234

Passo 7: Cadastre um plano, um médico e um paciente

Passo 8: Faça um agendamento e veja na listagem

DESAFIO BÔNUS: Adicione um botão 'Cancelar Consulta' na listagem, que altere o status para 'cancelado' no banco de dados.

Apêndice – Referência Rápida

Comandos MySQL Essenciais

CÓDIGO

```
-- Mostrar todos os bancos
SHOW DATABASES;

-- Selecionar um banco
USE medicare_db;

-- Ver tabelas do banco atual
SHOW TABLES;

-- Ver estrutura de uma tabela
DESCRIBE pacientes;

-- Ver todos os registros de uma tabela
SELECT * FROM pacientes;

-- Deletar todos os registros (cuidado!)
TRUNCATE TABLE agendamentos;
```

Erros Comuns e Soluções

Erro	Solução
ModuleNotFoundError: No module named 'pymysql'	Execute: pip install pymysql
Access denied for user 'root'@'localhost'	Verifique a senha no arquivo conexao.py
ImportError ao rodar view/arquivo.py	Execute apenas main.py, nunca os arquivos internos
Table 'medicare_db.pacientes' doesn't exist	Execute o SQL de criação das tabelas primeiro
OperationalError: (2003, "Can't connect to MySQL")	Verifique se o MySQL está rodando (XAMPP ou serviço)
ttkbootstrap not found	Execute: pip install ttkbootstrap

Parabéns! 

 Você concluiu o curso MediCare+!

Ao completar este projeto, você agora sabe:

- ☒ Organizar código com o padrão MVC
- ☒ Criar e manipular banco de dados MySQL com PyMySQL
 - ☒ Criar interfaces profissionais com ttkbootstrap
 - ☒ Construir um sistema desktop completo e real

"Programar não é só fazer funcionar. É organizar, estruturar e criar algo que possa crescer."